

Nginx 性能优化完全指南

课程概述

本课件系统讲解 Nginx 性能优化的各个层面，包括系统层优化、Nginx 核心配置优化、缓存优化、安全优化以及监控调优方法，帮助学员掌握构建高性能 Web 服务的核心技能。

第一部分：优化基础与方法论

1.1 性能优化的目标

性能优化的核心目标是在有限的硬件资源下，最大化系统的吞吐量，同时保持较低的响应延迟和较高的稳定性。具体而言，我们关注以下几个关键指标。

吞吐量 (Throughput) 表示系统在单位时间内能够处理的请求数量，通常用 QPS（每秒查询数）或 RPS（每秒请求数）来衡量。对于静态资源服务器，优化良好的 Nginx 单机可以达到数万甚至十万级别的 QPS。

响应时间 (Response Time) 表示从客户端发起请求到收到完整响应的时间。这个指标直接影响用户体验，一般要求控制在毫秒级别。响应时间又可细分为首字节时间 (TTFB) 和总响应时间。

并发连接数 (Concurrent Connections) 表示系统能够同时维持的客户端连接数量。Nginx 采用事件驱动架构，单个 worker 进程可以处理数千甚至数万个并发连接。

资源利用率 (Resource Utilization) 表示 CPU、内存、网络带宽、磁盘 I/O 等资源的使用效率。优化的目标是在不造成资源瓶颈的前提下，充分利用硬件能力。

1.2 优化方法论

性能优化应当遵循科学的方法论，避免盲目调参。首先需要建立性能基准，使用压测工具记录当前系统的各项性能指标。然后通过监控和分析工具定位性能瓶颈，确定优化方向。接下来针对瓶颈进行优化调整，每次只修改一个变量。最后重新测试验证优化效果，循环迭代直到达到预期目标。

优化顺序通常是从系统层开始，依次是内核参数、Nginx 配置、应用逻辑、架构调整。低层次的优化往往能带来更显著的效果，而且影响范围更广。

1.3 性能测试工具

在进行优化之前，需要掌握常用的性能测试工具。

wrk 是一款现代化的 HTTP 压测工具，基于多线程和事件驱动设计，能够产生极高的负载。

```
# 安装 wrk
sudo yum install -y git gcc make
git clone https://github.com/wg/wrk.git
cd wrk && make
sudo cp wrk /usr/local/bin/

# 基本压测命令
# -t 线程数, -c 并发连接数, -d 持续时间
wrk -t12 -c400 -d30s http://localhost/

# 带 Lua 脚本的高级测试
wrk -t12 -c400 -d30s -s post.lua http://localhost/api
```

ab (Apache Bench) 是 Apache 自带的压测工具，简单易用。

```
# 安装
sudo yum install -y httpd-tools

# 基本使用
# -n 总请求数, -c 并发数
ab -n 10000 -c 100 http://localhost/

# 带 Keep-Alive
ab -n 10000 -c 100 -k http://localhost/
```

第二部分：系统层优化

2.1 内核参数优化

操作系统内核参数对 Nginx 性能有决定性影响，特别是网络相关的参数。

文件描述符限制 是最常遇到的瓶颈之一。每个网络连接都需要占用一个文件描述符，默认限制通常是 1024，远远不能满足高并发需求。

```
# 查看当前限制
ulimit -n

# 临时修改（当前会话有效）
ulimit -n 65535

# 永久修改 - 编辑 limits.conf
sudo tee -a /etc/security/limits.conf << 'EOF'
# Nginx 进程的文件描述符限制
nginx soft nofile 65535
nginx hard nofile 65535
* soft nofile 65535
* hard nofile 65535
EOF

# 系统级别的文件描述符限制
sudo tee -a /etc/sysctl.conf << 'EOF'
# 系统最大文件描述符数量
fs.file-max = 2097152
```

EOF

```
# 使配置生效
sudo sysctl -p
```

TCP 网络参数 优化对于处理大量并发连接至关重要。

```
# 创建 Nginx 优化的内核参数配置文件
sudo tee /etc/sysctl.d/99-nginx-optimization.conf << 'EOF'
# =====
# TCP 连接优化
# =====

# SYN 队列长度，处理大量新连接时需要增大
net.ipv4.tcp_max_syn_backlog = 65535

# 系统同时保持 TIME_WAIT 状态的最大数量
net.ipv4.tcp_max_tw_buckets = 65535

# 允许重用 TIME_WAIT 状态的连接
net.ipv4.tcp_tw_reuse = 1

# 开启 SYN Cookie，防止 SYN 洪水攻击
net.ipv4.tcp_syncookies = 1

# FIN_WAIT_2 状态的超时时间（秒）
net.ipv4.tcp_fin_timeout = 15

# TCP Keep-Alive 探测间隔（秒）
net.ipv4.tcp_keepalive_time = 600
net.ipv4.tcp_keepalive_intvl = 30
net.ipv4.tcp_keepalive_probes = 3

# =====
# 网络缓冲区优化
# =====

# TCP 读写缓冲区大小（最小值、默认值、最大值）
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216
```

```
# 网络设备队列长度
net.core.netdev_max_backlog = 65535

# Socket 监听队列长度
net.core.somaxconn = 65535

# 网络核心读写缓冲区
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.core.rmem_default = 262144
net.core.wmem_default = 262144

# =====
# 端口范围优化
# =====

# 可用的本地端口范围
net.ipv4.ip_local_port_range = 1024 65535

# =====
# 内存优化
# =====

# 减少 swap 使用倾向
vm.swappiness = 10

# 文件系统缓存压力
vm.vfs_cache_pressure = 50
EOF

# 应用配置
sudo sysctl -p /etc/sysctl.d/99-nginx-optimization.conf
```

2.2 磁盘 I/O 优化

磁盘 I/O 是静态资源服务的关键瓶颈。选择合适的文件系统和调度算法可以显著提升性能。

```
# 查看当前磁盘调度算法
cat /sys/block/sda/queue/scheduler

# 对于 SSD, 建议使用 noop 或 deadline
echo noop | sudo tee /sys/block/sda/queue/scheduler

# 永久修改 (GRUB 配置)
# 编辑 /etc/default/grub, 在 GRUB_CMDLINE_LINUX 中添加:
# elevator=noop

# 更新 GRUB
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

文件系统挂载选项 也会影响性能。

```
# 编辑 /etc/fstab, 添加性能优化选项
# noatime - 不更新访问时间, 减少写操作
# nodiratime - 不更新目录访问时间
# 示例:
# /dev/sda1 /data ext4 defaults,noatime,nodiratime 0 2
```

2.3 网卡优化

对于高流量场景, 网卡配置同样重要。

```
# 查看网卡信息
ethtool eth0

# 开启网卡多队列 (如果支持)
ethtool -L eth0 combined 8

# 增大网卡环形缓冲区
ethtool -G eth0 rx 4096 tx 4096

# 开启 GRO (Generic Receive Offload)
ethtool -K eth0 gro on

# 开启 TSO (TCP Segmentation Offload)
```

```
ethtool -K eth0 tso on
```

第三部分：Nginx 核心配置优化

3.1 进程与连接优化

Nginx 的进程模型采用 Master-Worker 架构，合理配置进程参数是优化的基础。

```
# /etc/nginx/nginx.conf

# =====
# 全局配置
# =====

# Worker 进程数，通常设置为 CPU 核心数
# auto 表示自动检测
worker_processes auto;

# Worker 进程优先级，-20 到 20，越小优先级越高
worker_priority -10;

# 每个 Worker 进程能打开的最大文件描述符数
worker_rlimit_nofile 65535;

# 将 Worker 进程绑定到特定 CPU 核心，提高缓存命中率
# 4 核 CPU 的绑定示例
worker_cpu_affinity 0001 0010 0100 1000;
# 或者使用 auto 自动绑定
worker_cpu_affinity auto;

# 错误日志级别
error_log /var/log/nginx/error.log warn;

# PID 文件位置
pid /run/nginx.pid;

# =====
# 事件模块配置
# =====
```

```
events {
    # 使用 epoll 事件模型 (Linux 高性能必选)
    use epoll;

    # 每个 Worker 进程的最大并发连接数
    # 总并发数 = worker_processes × worker_connections
    worker_connections 65535;

    # 允许一次接受多个新连接
    multi_accept on;

    # 开启互斥锁，避免惊群效应
    accept_mutex off;
}
```

3.2 HTTP 核心配置优化

HTTP 模块的配置直接影响请求处理效率。

```
http {
    # =====
    # 基础配置
    # =====

    # 隐藏 Nginx 版本号，提高安全性
    server_tokens off;

    # 字符集
    charset utf-8;

    # MIME 类型
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # =====
    # 日志优化
    # =====

    # 自定义日志格式，添加性能相关字段
```



```
log_format main '$remote_addr - $remote_user [$time_local]
"$request" '
                '$status $body_bytes_sent "$http_referer" '
                '"$http_user_agent" "$http_x_forwarded_for" '
                '$request_time $upstream_response_time';

# 高并发场景可考虑关闭访问日志或异步写入
# access_log off;
access_log /var/log/nginx/access.log main buffer=64k flush=5s;

# 错误日志缓冲
error_log /var/log/nginx/error.log warn;

# 打开文件日志错误缓存
open_log_file_cache max=1000 inactive=20s valid=1m min_uses=2;

# =====
# 文件传输优化
# =====

# 开启 sendfile, 零拷贝传输文件
sendfile on;

# 配合 sendfile 使用, 优化网络包发送
tcp_nopush on;

# 禁用 Nagle 算法, 减少延迟
tcp_nodelay on;

# =====
# 缓冲区优化
# =====

# 客户端请求体缓冲区
client_body_buffer_size 16k;

# 客户端请求头缓冲区
client_header_buffer_size 4k;

# 大请求头缓冲区
large_client_header_buffers 4 32k;
```

```
# 客户端请求体最大大小
client_max_body_size 100m;

# =====
# 超时优化
# =====

# 客户端请求头超时
client_header_timeout 15s;

# 客户端请求体超时
client_body_timeout 15s;

# 响应发送超时
send_timeout 30s;

# Keep-Alive 超时
keepalive_timeout 65s;

# 单个 Keep-Alive 连接的最大请求数
keepalive_requests 1000;

# 服务器接收请求的超时
reset_timedout_connection on;

# =====
# 哈希表优化
# =====

# 服务器名称哈希表
server_names_hash_bucket_size 128;
server_names_hash_max_size 1024;

# 变量哈希表
variables_hash_max_size 2048;
variables_hash_bucket_size 128;

# 类型哈希表
types_hash_max_size 2048;
types_hash_bucket_size 128;

# =====
```

```

# 文件缓存优化
# =====

# 打开文件描述符缓存
open_file_cache max=100000 inactive=20s;
open_file_cache_valid 60s;
open_file_cache_min_uses 2;
open_file_cache_errors on;

# 其他配置...
include /etc/nginx/conf.d/*.conf;
}

```

3.3 静态资源优化

静态资源的优化是提升用户体验的关键。

```

server {
    listen 80;
    server_name static.example.com;
    root /data/static;

    # =====
    # Gzip 压缩配置
    # =====

    # 开启 Gzip
    gzip on;

    # 压缩级别 1-9, 推荐 4-6
    gzip_comp_level 5;

    # 最小压缩文件大小
    gzip_min_length 1024;

    # 压缩缓冲区
    gzip_buffers 16 8k;

    # 压缩的 MIME 类型
    gzip_types text/plain

```

```
text/css
text/javascript
text/xml
application/json
application/javascript
application/xml
application/xml+rss
application/x-javascript
image/svg+xml;
```

对代理请求也进行压缩

```
gzip_proxied any;
```

在响应头中添加 Vary: Accept-Encoding

```
gzip_vary on;
```

禁用对 IE6 的压缩

```
gzip_disable "MSIE [1-6]\.";
```

```
# =====
```

静态资源缓存

```
# =====
```

图片资源

```
location ~* \.(jpg|jpeg|png|gif|ico|webp|svg)$ {
    expires 30d;
    add_header Cache-Control "public, immutable";
    access_log off;
```

尝试提供 WebP 版本

```
try_files $uri$webp_suffix $uri =404;
```

```
}
```

CSS 和 JavaScript

```
location ~* \.(css|js)$ {
    expires 7d;
    add_header Cache-Control "public";
    access_log off;
```

```
}
```

字体文件

```
location ~* \.(woff|woff2|ttf|eot|otf)$ {
```

```

    expires 365d;
    add_header Cache-Control "public, immutable";
    access_log off;

    # 允许跨域
    add_header Access-Control-Allow-Origin "*";
}

# HTML 文件
location ~* \.html$ {
    expires 1h;
    add_header Cache-Control "public, must-revalidate";
}

# =====
# 防盗链配置
# =====

location /images/ {
    valid_referers none blocked server_names
                  *.example.com example.com;

    if ($invalid_referer) {
        return 403;
    }
}
}

```

3.4 Brotli 压缩

Brotli 是比 Gzip 更高效的压缩算法，压缩率提升约 20–30%。

```

# 安装 Brotli 模块（需要重新编译 Nginx 或使用动态模块）
# 以下是动态模块加载方式

# 首先确认 Nginx 版本和编译参数
nginx -V

# 下载并编译 Brotli 模块（这里简化展示概念）
# git clone https://github.com/google/nginx_brotli.git

```

```
# 然后重新编译 Nginx 添加 --add-dynamic-module=../ngx_brotli
# 加载 Brotli 模块
load_module modules/ngx_http_brotli_filter_module.so;
load_module modules/ngx_http_brotli_static_module.so;

http {
    # Brotli 压缩配置
    brotli on;
    brotli_comp_level 6;
    brotli_min_length 1024;
    brotli_types text/plain
                  text/css
                  text/javascript
                  application/json
                  application/javascript
                  application/xml
                  image/svg+xml;

    # 优先提供预压缩的 .br 文件
    brotli_static on;
}
```

第四部分：代理与负载均衡优化

4.1 反向代理优化

当 Nginx 作为反向代理时，需要特别关注代理相关的配置。

```
http {
    # =====
    # 代理缓冲区配置
    # =====

    # 开启代理缓冲
    proxy_buffering on;

    # 代理响应头缓冲区
    proxy_buffer_size 8k;
```

```
# 代理响应体缓冲区
proxy_buffers 32 8k;

# 繁忙时的缓冲区大小
proxy_busy_buffers_size 64k;

# 临时文件相关
proxy_temp_file_write_size 64k;
proxy_max_temp_file_size 1024m;

# =====
# 代理超时配置
# =====

# 与后端建立连接的超时时间
proxy_connect_timeout 10s;

# 向后端发送请求的超时时间
proxy_send_timeout 60s;

# 从后端读取响应的超时时间
proxy_read_timeout 60s;

# =====
# 代理头配置
# =====

# 传递真实客户端信息
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;

# HTTP 版本
proxy_http_version 1.1;

# 保持连接
proxy_set_header Connection "";

}
```

4.2 负载均衡优化

负载均衡的核心是合理分配流量，同时保证后端服务的健康。

```
http {  
    # =====  
    # Upstream 定义  
    # =====  
  
    upstream backend_cluster {  
        # 负载均衡算法  
        # least_conn - 最少连接  
        # ip_hash - IP 哈希  
        # hash $request_uri consistent - 一致性哈希  
        least_conn;  
  
        # 后端服务器配置  
        # weight: 权重  
        # max_fails: 最大失败次数  
        # fail_timeout: 失败后的屏蔽时间  
        # max_conns: 最大并发连接数  
        server 192.168.1.10:8080 weight=5 max_fails=3  
fail_timeout=30s max_conns=1000;  
        server 192.168.1.11:8080 weight=5 max_fails=3  
fail_timeout=30s max_conns=1000;  
        server 192.168.1.12:8080 weight=3 max_fails=3  
fail_timeout=30s max_conns=800;  
  
        # 备用服务器，仅在主服务器全部不可用时启用  
        server 192.168.1.13:8080 backup;  
  
        # 标记为永久不可用  
        # server 192.168.1.14:8080 down;  
  
        # 长连接配置  
        keepalive 100;  
        keepalive_timeout 60s;  
        keepalive_requests 1000;  
  
        # 队列配置（商业版功能）  
        # queue 100 timeout=70s;
```



```

}

server {
    listen 80;
    server_name api.example.com;

    location / {
        proxy_pass http://backend_cluster;

        # 使用 HTTP/1.1 并保持连接
        proxy_http_version 1.1;
        proxy_set_header Connection "";

        # 失败重试
        proxy_next_upstream error timeout http_500 http_502
http_503 http_504;
        proxy_next_upstream_timeout 10s;
        proxy_next_upstream_tries 3;

        # 重试时不重复发送请求体
        proxy_request_buffering on;
    }
}

```

4.3 健康检查

Nginx 开源版的健康检查依赖被动方式，商业版支持主动健康检查。

```

# 开源版被动健康检查（基于 max_fails 和 fail_timeout）
upstream backend {
    server 192.168.1.10:8080 max_fails=3 fail_timeout=30s;
    server 192.168.1.11:8080 max_fails=3 fail_timeout=30s;
}

# 商业版主动健康检查示例
upstream backend {
    zone backend 64k;

    server 192.168.1.10:8080;

```

```
server 192.168.1.11:8080;

# 主动健康检查
health_check interval=5s fails=3 passes=2 uri=/health;
}
```

对于开源版，可以使用第三方模块 `nginx_upstream_check_module` 实现主动健康检查。

第五部分：缓存优化

5.1 代理缓存配置

Nginx 的代理缓存可以显著减少后端压力并加速响应。

```
http {
    # =====
    # 缓存路径定义
    # =====

    # 定义缓存存储路径
    # levels: 目录层级
    # keys_zone: 共享内存区域名称和大小
    # max_size: 缓存最大大小
    # inactive: 未访问数据的过期时间
    # use_temp_path: 是否使用临时目录
    proxy_cache_path /data/nginx/cache
                    levels=1:2
                    keys_zone=my_cache:100m
                    max_size=10g
                    inactive=60m
                    use_temp_path=off;

    # 定义缓存 key
    proxy_cache_key "$scheme$request_method$host$request_uri";

    server {
        listen 80;
```

```
server_name www.example.com;

location / {
    proxy_pass http://backend;

    # 启用缓存
    proxy_cache my_cache;

    # 缓存有效期
    proxy_cache_valid 200 301 302 10m;
    proxy_cache_valid 404 1m;
    proxy_cache_valid any 5m;

    # 缓存最少使用次数
    proxy_cache_min_uses 2;

    # 缓存锁，防止缓存击穿
    proxy_cache_lock on;
    proxy_cache_lock_timeout 5s;
    proxy_cache_lock_age 5s;

    # 后端故障时使用过期缓存
    proxy_cache_use_stale error timeout updating
        http_500 http_502 http_503
http_504;

    # 后台更新缓存
    proxy_cache_background_update on;

    # 缓存重验证
    proxy_cache_revalidate on;

    # 添加缓存状态头
    add_header X-Cache-Status $upstream_cache_status;

    # 设置缓存的 Vary 响应头
    proxy_ignore_headers Vary;
}

# 不缓存动态内容
location ~ \.(php|js|plasp)$ {
    proxy_pass http://backend;
```

```

        proxy_cache off;
    }

    # 根据 Cookie 跳过缓存
    location /user/ {
        proxy_pass http://backend;

        set $skip_cache 0;

        if ($http_cookie ~* "session_id") {
            set $skip_cache 1;
        }

        proxy_cache my_cache;
        proxy_cache_bypass $skip_cache;
        proxy_no_cache $skip_cache;
    }
}

```

5.2 FastCGI 缓存

对于 PHP 等 FastCGI 应用，可以使用 FastCGI 缓存。

```

http {
    # FastCGI 缓存路径
    fastcgi_cache_path /data/nginx/fastcgi_cache
        levels=1:2
        keys_zone=php_cache:50m
        max_size=5g
        inactive=60m;

    # 缓存 key
    fastcgi_cache_key "$scheme$request_method$host$request_uri";

    server {
        listen 80;
        server_name www.example.com;

        location ~ /\.php$ {

```

```

        fastcgi_pass unix:/run/php/php-fpm.sock;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME
$document_root$fastcgi_script_name;
        include fastcgi_params;

# 启用 FastCGI 缓存
        fastcgi_cache php_cache;
        fastcgi_cache_valid 200 301 302 1h;
        fastcgi_cache_valid 404 10m;

# 缓存绕过条件
        fastcgi_cache_bypass $cookie_nocache $arg_nocache;
        fastcgi_no_cache $cookie_nocache $arg_nocache;

# 添加缓存状态头
        add_header X-FastCGI-Cache $upstream_cache_status;
    }
}
}

```

5.3 浏览器缓存策略

合理的浏览器缓存策略可以减少重复请求。

```

server {
    listen 80;
    server_name www.example.com;

# 版本化资源：长期缓存
# 文件名包含哈希值，如 app.a1b2c3d4.js
    location ~* \.[a-f0-9]{8}\.(js|css|png|jpg|gif|ico|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        access_log off;
    }

# 非版本化资源：短期缓存
    location ~* \.(js|css|png|jpg|gif|ico)$ {
        expires 7d;
    }
}

```

```

    add_header Cache-Control "public, must-revalidate";
    etag on;
}

# HTML 文件：协商缓存
location ~* \.html$ {
    expires -1;
    add_header Cache-Control "no-cache, must-revalidate";
    etag on;
}

# API 响应：不缓存
location /api/ {
    add_header Cache-Control "no-store, no-cache, must-
revalidate, proxy-revalidate";
    add_header Pragma "no-cache";
    expires 0;
}
}

```

第六部分：SSL/TLS 优化

6.1 SSL 会话优化

SSL/TLS 握手是 HTTPS 的性能瓶颈，优化握手过程可以显著提升性能。

```

http {
    # =====
    # SSL 基础配置
    # =====

    # SSL 协议版本
    ssl_protocols TLSv1.2 TLSv1.3;

    # 加密套件（按优先级排列）
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-
SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;

```

```
# 优先使用服务器端加密套件
ssl_prefer_server_ciphers on;

# =====
# SSL 会话缓存
# =====

# 会话缓存类型和大小
# shared: 所有 worker 共享
# 1m 大约可存储 4000 个会话
ssl_session_cache shared:SSL:50m;

# 会话超时时间
ssl_session_timeout 1d;

# 禁用会话票据（如果需要完美前向保密）
# ssl_session_tickets off;

# 会话票据密钥（多服务器需保持一致）
ssl_session_ticket_key /etc/nginx/ssl/ticket.key;

# =====
# OCSP Stapling
# =====

# 开启 OCSP Stapling
ssl_stapling on;
ssl_stapling_verify on;

# 信任的 CA 证书链
ssl_trusted_certificate /etc/nginx/ssl/chain.pem;

# DNS 解析器（用于 OCSP 查询）
resolver 8.8.8.8 8.8.4.4 valid=300s;
resolver_timeout 5s;

# =====
# HTTP/2 配置
# =====

server {
    listen 443 ssl http2;
```

```

server_name www.example.com;

ssl_certificate /etc/nginx/ssl/cert.pem;
ssl_certificate_key /etc/nginx/ssl/key.pem;

# 针对 HTTP/2 的优化
http2_max_concurrent_streams 128;
http2_idle_timeout 3m;

# 其他配置...
}
}

```

6.2 TLS 1.3 优化

TLS 1.3 提供更快的握手和更强的安全性。

```

http {
    # 仅使用 TLS 1.3
    ssl_protocols TLSv1.3;

    # TLS 1.3 加密套件
    ssl_conf_command Ciphersuites
    TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM_S
    HA256;

    # 启用 0-RTT (早期数据)
    ssl_early_data on;

    server {
        listen 443 ssl http2;
        server_name www.example.com;

        # 防止 0-RTT 重放攻击
        location / {
            # 对于非幂等请求, 检查早期数据
            if ($ssl_early_data) {
                # 可以选择拒绝或添加标记
                add_header X-Early-Data $ssl_early_data;
            }
        }
    }
}

```



```
        proxy_pass http://backend;
    }
}
}
```

第七部分：安全优化

7.1 请求限制

防止恶意请求和 DDoS 攻击。

```
http {
    # =====
    # 请求速率限制
    # =====

    # 定义限制区域
    # $binary_remote_addr: 使用二进制 IP 地址，节省空间
    # zone: 共享内存区域
    # rate: 速率限制
    limit_req_zone $binary_remote_addr zone=req_limit:10m rate=10r/s;
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=5r/s;
    limit_req_zone $server_name zone=server_limit:10m rate=1000r/s;

    # 限制状态码
    limit_req_status 429;

    # =====
    # 连接数限制
    # =====

    # 定义连接限制区域
    limit_conn_zone $binary_remote_addr zone=conn_limit:10m;
    limit_conn_zone $server_name zone=server_conn:10m;

    # 限制状态码
    limit_conn_status 429;
```

```

server {
    listen 80;
    server_name www.example.com;

    # 全局连接限制
    limit_conn conn_limit 50;
    limit_conn server_conn 10000;

    location / {
        # 请求速率限制
        # burst: 允许突发请求数
        # nodelay: 不延迟处理突发请求
        limit_req zone=req_limit burst=20 nodelay;

        proxy_pass http://backend;
    }

    location /api/ {
        # API 接口更严格的限制
        limit_req zone=api_limit burst=10 nodelay;
        limit_conn conn_limit 20;

        proxy_pass http://backend;
    }

    # 登录接口特别限制
    location /api/login {
        limit_req zone=api_limit burst=5;
        limit_conn conn_limit 5;

        proxy_pass http://backend;
    }
}

```

7.2 安全响应头

添加安全相关的 HTTP 响应头。

```

server {

```

```

listen 443 ssl http2;
server_name www.example.com;

# =====
# 安全响应头
# =====

# 防止点击劫持
add_header X-Frame-Options "SAMEORIGIN" always;

# 防止 MIME 类型嗅探
add_header X-Content-Type-Options "nosniff" always;

# XSS 过滤器
add_header X-XSS-Protection "1; mode=block" always;

# 内容安全策略
add_header Content-Security-Policy "default-src 'self'; script-
src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-
inline';" always;

# 引用策略
add_header Referrer-Policy "strict-origin-when-cross-origin"
always;

# 权限策略
add_header Permissions-Policy "geolocation=(), microphone=(),
camera=()" always;

# HSTS (强制 HTTPS)
add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload" always;

# 隐藏服务器信息
server_tokens off;
more_clear_headers Server;

location / {
    proxy_pass http://backend;
}
}

```

7.3 访问控制

基于 IP 和其他条件的访问控制。

```
http {  
    # =====  
    # IP 黑白名单  
    # =====  
  
    # 使用 geo 模块定义 IP 分类  
    geo $blocked_ip {  
        default 0;  
        10.0.0.0/8 1;  
        192.168.0.0/16 0;  
        # 可以包含外部文件  
        include /etc/nginx/blocked_ips.conf;  
    }  
  
    # 使用 map 定义白名单  
    map $remote_addr $is_allowed {  
        default 0;  
        192.168.1.0/24 1;  
        10.0.0.100 1;  
    }  
  
    server {  
        listen 80;  
        server_name admin.example.com;  
  
        # 检查是否被封禁  
        if ($blocked_ip) {  
            return 403;  
        }  
  
        # 管理后台仅允许白名单 IP  
        location /admin/ {  
            allow 192.168.1.0/24;  
            allow 10.0.0.100;  
            deny all;  
  
            proxy_pass http://backend;
```

```
}

# 屏蔽特定 User-Agent
if ($http_user_agent ~* (bot|spider|crawler|scanner)) {
    return 403;
}

location / {
    proxy_pass http://backend;
}
}
```

第八部分：监控与调优

8.1 状态监控配置

开启 Nginx 状态页面用于监控。

```
server {
    listen 80;
    server_name localhost;

    # Nginx 内置状态页
    location /nginx_status {
        stub_status on;
        access_log off;
        allow 127.0.0.1;
        allow 192.168.1.0/24;
        deny all;
    }

    # 如果使用 VTS 模块
    location /status {
        vhost_traffic_status_display;
        vhost_traffic_status_display_format html;
        access_log off;
        allow 127.0.0.1;
        allow 192.168.1.0/24;
    }
}
```

```
        deny all;
    }
}
```

8.2 日志分析

通过日志分析发现性能问题。

```
#!/bin/bash
# 性能分析脚本

LOG_FILE="/var/log/nginx/access.log"

echo "===== "
echo "Nginx 性能分析报告"
echo "时间: $(date)"
echo "===== "

# 请求量统计
echo ""
echo "1. 请求量统计"
echo "-----"
echo "总请求数: $(wc -l < $LOG_FILE)"
echo "QPS (最后1分钟): $(tail -n 10000 $LOG_FILE | awk -v now=$(date +%s) 'BEGIN{count=0} {split($4,a,":"); gsub(/\[/, "", a[1]); count++} END{print count/60}')"

# 响应时间分布
echo ""
echo "2. 响应时间分析"
echo "-----"
awk '{print $NF}' $LOG_FILE | sort -n | awk '
BEGIN { count = 0; sum = 0 }
{ times[count++] = $1; sum += $1 }
END {
    print "平均响应时间: " sum/count "s"
    print "P50: " times[int(count*0.5)] "s"
    print "P90: " times[int(count*0.9)] "s"
    print "P99: " times[int(count*0.99)] "s"
    print "最大响应时间: " times[count-1] "s"
}
```

```

}
'

# 状态码分布
echo ""
echo "3. 状态码分布"
echo "-----"
awk '{print $9}' $LOG_FILE | sort | uniq -c | sort -rn

# 慢请求分析
echo ""
echo "4. 慢请求 TOP 10 (响应时间 > 1s)"
echo "-----"
awk '$NF > 1 {print $7, $NF}' $LOG_FILE | sort -k2 -rn | head -10

# 高频 IP
echo ""
echo "5. 访问量 TOP 10 IP"
echo "-----"
awk '{print $1}' $LOG_FILE | sort | uniq -c | sort -rn | head -10

# 高频 URL
echo ""
echo "6. 访问量 TOP 10 URL"
echo "-----"
awk '{print $7}' $LOG_FILE | sort | uniq -c | sort -rn | head -10

```

8.3 实时监控脚本

```

#!/bin/bash
# 实时监控脚本

while true; do
    clear
    echo "=====
    echo "Nginx 实时监控 - $(date '+%Y-%m-%d %H:%M:%S')"
    echo "=====

    # 获取 stub_status 数据
    STATUS=$(curl -s http://localhost/nginx_status)

```

```
ACTIVE=$(echo "$STATUS" | grep 'Active' | awk '{print $3}')
ACCEPTS=$(echo "$STATUS" | grep -A 1 'server' | tail -1 | awk
'{print $1}')
HANDLED=$(echo "$STATUS" | grep -A 1 'server' | tail -1 | awk
'{print $2}')
REQUESTS=$(echo "$STATUS" | grep -A 1 'server' | tail -1 | awk
'{print $3}')
READING=$(echo "$STATUS" | grep 'Reading' | awk '{print $2}')
WRITING=$(echo "$STATUS" | grep 'Reading' | awk '{print $4}')
WAITING=$(echo "$STATUS" | grep 'Reading' | awk '{print $6}')

echo ""
echo "连接状态:"
echo "  活跃连接数: $ACTIVE"
echo "  正在读取: $READING"
echo "  正在写入: $WRITING"
echo "  等待中: $WAITING"
echo ""
echo "累计统计:"
echo "  总接受连接: $ACCEPTS"
echo "  总处理连接: $HANDLED"
echo "  总请求数: $REQUESTS"
echo ""

# 进程状态
echo "Worker 进程:"
ps aux | grep 'nginx: worker' | grep -v grep | awk '{printf "
PID: %s  CPU: %s%%  MEM: %s%%\n", $2, $3, $4}'

# 系统资源
echo ""
echo "系统资源:"
echo "  CPU: $(top -bn1 | grep "Cpu(s)" | awk '{print $2}')"%"
echo "  内存: $(free -m | awk 'NR==2{printf "已用 %sMB / 总计 %sMB
(%.1f%%)", $3, $2, $3*100/$2}')"
echo "  连接数: $(ss -s | grep 'TCP:' | awk '{print $2}')"

sleep 2
done
```


第九部分：常见性能问题排查

9.1 问题诊断清单

问题现象	可能原因	排查方法	解决方案
连接被拒绝	文件描述符耗尽	<code>ulimit -n</code>	增大 <code>worker_rlimit_nofile</code>
响应慢	后端处理慢	查看 <code>upstream_response_time</code>	优化后端或增加缓存
502 错误	后端不可用	检查后端服务状态	健康检查和故障转移
504 错误	后端超时	查看超时配置	增大 <code>proxy_read_timeout</code>
CPU 高	压缩或 SSL 开销	<code>top</code> 查看进程	调整压缩级别或启用硬件加速
内存高	缓冲区过大	检查 <code>buffer</code> 配置	调整缓冲区大小
磁盘 I/O 高	日志或缓存写入	<code>iostat</code> 分析	异步日志或 SSD

9.2 性能调优检查清单

- ☐ 系统层优化

☐ 文件描述符限制调整

☐ TCP 内核参数优化

☐ 网卡参数优化

☐ 磁盘 I/O 优化
- ☐ Nginx 核心配置

☐ `worker_processes` 设置为 CPU 核心数

☐ `worker_connections` 根据预期并发调整

☐ `worker_cpu_affinity` 绑定 CPU

- ☐ 使用 `epoll` 事件模型
- ☐ HTTP 优化
 - ☐ 开启 `sendfile`
 - ☐ 开启 `tcp_nopush` 和 `tcp_nodelay`
 - ☐ 配置合理的超时时间
 - ☐ 开启 `keepalive`
 - ☐ 配置 `open_file_cache`
- ☐ 压缩优化
 - ☐ 开启 `Gzip` 压缩
 - ☐ 配置合理的压缩级别
 - ☐ 静态文件预压缩
- ☐ 缓存优化
 - ☐ 配置代理缓存
 - ☐ 配置浏览器缓存
 - ☐ 使用 `CDN`
- ☐ SSL 优化
 - ☐ 使用 `TLS 1.3`
 - ☐ 开启 `SSL` 会话缓存
 - ☐ 开启 `OCSP Stapling`
 - ☐ 使用 `HTTP/2`
- ☐ 安全优化
 - ☐ 配置请求限制
 - ☐ 添加安全响应头
 - ☐ 配置访问控制
- ☐ 监控告警
 - ☐ 开启状态监控
 - ☐ 配置日志分析
 - ☐ 设置性能告警

第十部分：最佳实践总结

10.1 配置文件组织

```
/etc/nginx/
├── nginx.conf          # 主配置文件
├── conf.d/             # 虚拟主机配置
│   ├── default.conf
│   ├── api.example.com.conf
│   └── static.example.com.conf
├── snippets/          # 可重用配置片段
│   ├── ssl-params.conf
│   ├── security-headers.conf
│   ├── proxy-params.conf
│   └── gzip.conf
├── upstreams/         # 上游服务器配置
│   └── backend.conf
└── ssl/               # SSL 证书
    ├── example.com.crt
    └── example.com.key
```

10.2 生产环境推荐配置模板

```
# /etc/nginx/nginx.conf

user nginx;
worker_processes auto;
worker_cpu_affinity auto;
worker_priority -10;
worker_rlimit_nofile 65535;

error_log /var/log/nginx/error.log warn;
pid /run/nginx.pid;

events {
    use epoll;
    worker_connections 65535;
    multi_accept on;
    accept_mutex off;
}

http {
    # 基础配置
    include /etc/nginx/mime.types;
```

```
default_type application/octet-stream;
charset utf-8;
server_tokens off;

# 日志配置
log_format main '$remote_addr - $remote_user [$time_local]
"$request" '
                '$status $body_bytes_sent "$http_referer" '
                '"$http_user_agent" $request_time
$upstream_response_time';

access_log /var/log/nginx/access.log main buffer=64k flush=5s;

# 文件传输
sendfile on;
tcp_nopush on;
tcp_nodelay on;

# 超时配置
keepalive_timeout 65;
keepalive_requests 1000;
client_header_timeout 15;
client_body_timeout 15;
send_timeout 30;
reset_timedout_connection on;

# 缓冲区配置
client_body_buffer_size 16k;
client_header_buffer_size 4k;
large_client_header_buffers 4 32k;
client_max_body_size 100m;

# 文件缓存
open_file_cache max=100000 inactive=20s;
open_file_cache_valid 60s;
open_file_cache_min_uses 2;
open_file_cache_errors on;

# 包含其他配置
include /etc/nginx/snippets/gzip.conf;
include /etc/nginx/snippets/ssl-params.conf;
include /etc/nginx/upstreams/*.conf;
```

```
include /etc/nginx/conf.d/*.conf;  
}
```

10.3 优化效果评估

进行优化后，应当重新进行压力测试，对比优化前后的指标变化。关键评估指标包括：QPS 提升百分比、平均响应时间降低幅度、P99 响应时间变化、CPU 和内存使用率变化、以及错误率变化。

建议建立持续的性能监控机制，定期回顾和优化，确保系统始终保持在最佳状态。